Software Requirements and Design Document

For

Group <5>

Version 1.0

Authors:
Olivia Reiter
Julissa Su
Brooklyn Metzger
Yulissa Fu
Aleksandar “The Great” Stavreski
Alexander Hsieh

1. Overview (5 points)

Give a general overview of the system in 1-2 paragraphs (similar to the one in the project proposal).

The F1 Race Predictor is a web application that leverages machine learning to predict Formula 1 race outcomes using historical performance data, weather conditions, and race strategy parameters. The system provides users with real-time predictions for upcoming races, current driver and constructor championship standings, and a customizable race simulation tool that allows adjustment of variables such as circuit selection, weather, and tire strategy. The application also includes a comprehensive historical data center covering race results from 2010 to present, and a news feed aggregating current F1 updates.

The system is built on a React/TypeScript frontend, a Python FastAPI backend, and a PostgreSQL database, with machine learning components developed in Python using scikit-learn and pandas. It is designed as an open-access platform requiring no user authentication, prioritizing ease of use and data transparency over competitive or social features — distinguishing it from existing F1 prediction apps that focus primarily on fantasy leagues and point-based competitions.

2. Functional Requirements (10 points)

*List the **functional requirements** in sentences identified by numbers and for each requirement state if it is of high, medium, or low priority. Each functional requirement is something that the system shall do. Include all the details required such that there can be no misinterpretations of the requirements when read. Be very specific about what the system needs to do (not how, just what). You may provide a brief design rationale for any requirement which you feel requires explanation for how and/or why the requirement was derived.*

FR-X: ML Prediction Service (High Priority)

- The system shall predict race winner probabilities using a trained machine learning model with minimum 85% accuracy.

FR-X+1: Feature Calculation (High Priority)

- The system shall calculate driver statistics (win rate, recent form, team average) from historical race data for model input.

FR-X+2: Model Versioning (Medium Priority)

- The system shall track and log the ML model version in use.

FR-X+3: Dashboard Display (High Priority)

- The system shall display the upcoming F1 race information, including the race name circuit location, and a live countdown.

FR-X+4: Real-time UI Updates (Medium Priority)

- The frontend Dashboard shall automatically refresh race data every five minutes in the background.

FR-X+5: API Client Data Retrieval (High Priority)

- The system shall retrieve F1 data (such as next race) via centralized API client layer.

FR-X+6: Historical Race Data Retrieval (High Priority)

- The system shall fetch historical F1 race schedules, results, driver standings, and constructor standings from Jolpica F1

FR-X+7: Race Schedule Data (High Priority)

- The system shall retrieve current and upcoming race schedules.

FR-X+8: Database Population (High Priority)

- The system shall populate the database with race schedules and circuit information, preventing duplicate entries through validation checks on year and round number combinations

FR-X+9: Idempotent Seeding (Medium Priority)

- The system shall allow database seeding scripts to run multiple times safely, updating existing records rather than duplicating

3. Non-functional Requirements (10 points)

*List the **non-functional requirements** of the system (any requirement referring to a property of the system, such as security, safety, software quality, performance, reliability, etc.) You may provide a brief rationale for any requirement which you feel requires explanation as to how and/or why the requirement was derived.*

NFR-X: Prediction Performance (High Priority)
- The system shall generate race predictions without noticeable delay (<2 seconds).

NFR-X+1: Model Reliability (High Priority)
- The system shall handle model loading failures gracefully with clear error messages.

NFR-X+2: Model Accuracy (High Priority)
- The ML model shall maintain minimum 85% accuracy on test data.

NFR-X+3: Usability / Responsiveness (Low Priority)
- The frontend interface must be responsive, and its layout must remain readable on both desktop and mobile devices.

NFR-X+4: Reliability (High Priority)
- The frontend API must gracefully handle network errors, attempting up to three times before displaying a clear error message.

NFR-X+5: API Rate Limiting (High Priorirty)
- The system shall enforce rate limiting of 2 API calls per second to prevent service abuse

NFR-X+6: Seeding Performance (Medium Priority)
- The system shall complete database seeding of one F1 season in under 60 seconds

NFR-X+7: Cache Storage Efficiency (Low Priority)
- The system shall store cached API responses in an efficient file-based structure

4. Use Case Diagram (10 points)

*This section presents the **use case diagram** and the **textual descriptions** of the use cases for the system under development. The use case diagram should contain all the use cases and*

relationships between them needed to describe the functionality to be developed. If you discover new use cases between two increments, update the diagram for your future increments.
Textual descriptions of use cases: ***For the first increment, the textual descriptions for the use cases are not required.*** *However, the textual descriptions for all use cases discovered for your system are required for the second and third iterations.*

5. Class Diagram and/or Sequence Diagrams (15 points)

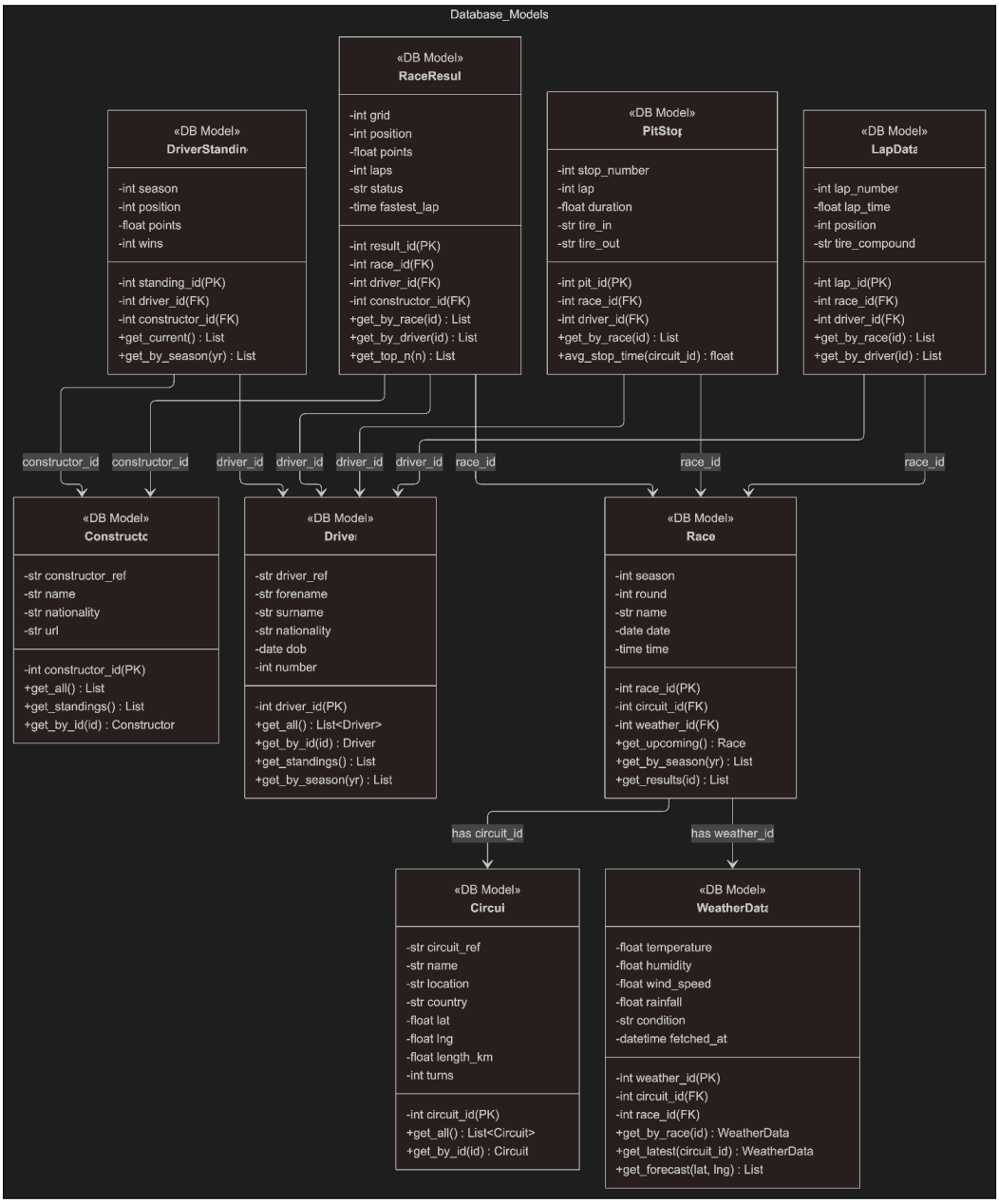
This section presents a high-level overview of the anticipated system architecture using a ***class diagram*** *and/or* ***sequence diagrams****.*

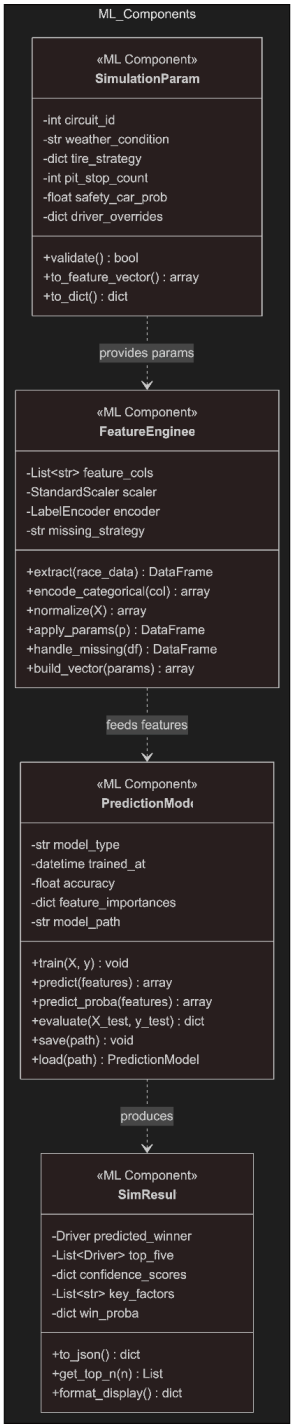
If the main ***paradigm*** *used in your project is* ***Object Oriented*** *(i.e., you have classes or something that acts similar to classes in your system), then draw the* ***Class Diagram of the entire system and Sequence Diagrams for the three (3) most important use cases in your system.***

If the main ***paradigm*** *in your system is* ***<u>not</u> Object Oriented*** *(i.e., you* ***<u>do not</u>*** *have classes or anything similar to classes in your system) then only draw* ***Sequence Diagrams****,* ***but for <u>all</u> the use cases of your system.*** *In this case, we will use a modified version of Sequence Diagrams, where instead of objects, the lifelines will represent the <u>functions</u> in the system involved in the action sequence.*

Class Diagrams *show the* ***fundamental objects/classes*** *that must be modeled with the system to satisfy its requirements and* ***the relationships*** *between them. Each class rectangle on the diagram* ***must also include the attributes and the methods of the class*** *(they can be refined between increments). All the* ***relationships between classes and their multiplicity*** *must be shown on the class diagram.*

A ***Sequence Diagram*** *simply depicts* ***interaction between objects*** *(or* ***functions -*** *in our case - for non-OOP systems) in a sequential order, i.e. the order in which these interactions take place. Sequence diagrams describe how and in what order the objects in a system function*





Class diagram pictured above.

Sequence Diagram (for "Predict Race Winner" use case)
User → API → Database → F1Predictor → Model → API → User

1. User requests prediction (driver, grid position)
2. API queries database (driver stats, team avg, recent form)
3. API calls F1Predictor.predict_race_winner()
4. F1Predictor formats input as DataFrame
5. Model.predict_proba() returns probability
6. F1Predictor formats result
7. API returns prediction to User

Sequence Diagram (for "Fetch Next Race data" use case)
User → React UI → API Client → FastAPI → PostgreSQL → FastAPI → API Client → React UI

1. User navigates to Dashboardhome page
2. React UI component calls fetchnextrace() from the API Client.
3. API Client sends an HTTP GET request to FastAPI
4. FastAPI queries PostgreSQL
5. PostgreSQL returns the specific race data
6. FastAPI formats the data into JSON and returns it to the API Client.
7. React UI updates its state and displays the race info to the user.

Sequence Diagram (for “Seed Race Data” use case)
Administrator → seed_races.py → JolpicaF1Client → Jolpica API → JolpicaF1Client → CacheManager → Database → Administrator

1. Administrator executes seed_races.py
2. Script initializes JolpicaF1Client and calls get_race_schedule
3. JolpicaF1Client checks CacheManager for cached data
4. If not cached, JolpicaF1Client calls Jolpica API
5. RateLimiter enforces 2 calls/second delay
6. API returns race schedule JSON
7. Script transforms data using transform_race()
8. Script inserts/updates races in Database
9. Database validates for duplicates
10. Script confirms success to Administrator

6. Operating Environment (5 points)

Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.

- Python 3.12+ (for ML model execution, backend execution, API client operations)
- scikit-learn 1.3+ (ML library)
- pandas 2.0+ (data processing)
- Node.js 18+ (for frontend and Vite)
- Modern Web Browser (FireFox, Safari, Chrome, etc) for client-side execution of React.
- PostgreSQL 16 (database server)
- psycopg2 (PostgreSQL adapter)
- Alembic (migration management)
- SQLAlchemy 2.0+ (ORM and database management)
- Pydantic v2 (data validation and schema enforcement)
- Uvicorn (ASGI server for running FastAPI on localhost:8000)
- Pytest 9.0+ (testing framework for integration tests)
- Python-dotenv 1.2+ (environment variable management)

7. Assumptions and Dependencies (5 points)

List any assumed factors (as opposed to known facts) that could affect the requirements stated in this document. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project.

Assumptions:

- Historical F1 data (2010-2024) available and accurate
- Driver/team performance patterns remain relatively stable year-over-year
- Jolpica F1 API remains available and free with no authentication requirements

Dependencies:

- scikit-learn library for model execution
- PostgreSQL database for feature calculation
- Jolpica F1 API for historical training data
- React for generating the user interface.
- Vite for handling the build process.
- Axios for handling HTTP communication between the frontend and backend.
- React Router for handling client-side navigation.
- Tailwind CSS for handling responsive design and styling.